

3.To Apply Escape Sequences, Store Command History, Navigate Previous Commands, Navigate Next Commands, Update Input Buffer, Test Recall Functionality.

To Allocate Buffers Dynamically, Resize Arrays, Prevent Buffer Overflow, Manage Linked Lists, Release Memory Correctly, Verify with Valgrind. \

```
GNU nano 8.7.1 skill3.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_INPUT 1024
#define MAX_TOKENS 100

int main()
{
    char input[MAX_INPUT];
    char *tokens[MAX_TOKENS];
    int tokenCount;

    printf("===== Mini Shell Parser =====\n");
    printf("Type 'exit' to quit.\n\n");

    while (1)
    {
        printf("MiniShell> ");

        if (fgets(input, sizeof(input), stdin) == NULL)
            break;

        input[strcspn(input, "\n")] = '\0';

        if (strlen(input) == 0)
        {
            printf("Empty command.\n");
            continue;
        }

        if (strcmp(input, "exit") == 0)
            break;

        tokenCount = 0;

        char *token = strtok(input, " \t");

        while (token != NULL && tokenCount < MAX_TOKENS - 1)
        {
            tokens[tokenCount++] = token;
            token = strtok(NULL, " \t");
        }
    }
}
```

```

GNU nano 8.7.1 skill3.c *
while (token != NULL && tokenCount < MAX_TOKENS - 1)
{
    tokens[tokenCount++] = token;
    token = strtok(NULL, " \t");
}

tokens[tokenCount] = NULL;

printf("\nParsed Tokens:\n");

for (int i = 0; i < tokenCount; i++)
{
    printf("Token %d : %s\n", i + 1, tokens[i]);
}

printf("\nCommand Structure:\n");

if (tokenCount > 0)
{
    printf("Command : %s\n", tokens[0]);

    if (tokenCount > 1)
    {
        printf("Arguments:\n");

        for (int i = 1; i < tokenCount; i++)
        {
            printf(" Arg %d : %s\n", i, tokens[i]);
        }
    }
    else
    {
        printf("No Arguments\n");
    }
}

printf("-----\n");

printf("Shell Closed.\n");

return 0;
}

```

```

thumu_hemanth_kumar@Hemanths-TUF-Laptop:~$ ls
2520090074_skill OSSkill os practical1 practical1.c practical2.c program1 program2
OSPractical destination.txt practical1.c practical practical2 practical4.c program1.c sample.t
thumu_hemanth_kumar@Hemanths-TUF-Laptop:~$ cd OSSkill
thumu_hemanth_kumar@Hemanths-TUF-Laptop:~/OSSkill$ nano skill3.c
thumu_hemanth_kumar@Hemanths-TUF-Laptop:~/OSSkill$ gcc skill3.c -o skill3
thumu_hemanth_kumar@Hemanths-TUF-Laptop:~/OSSkill$ ./skill3
===== Mini Shell Parser =====
Type 'exit' to quit.

MiniShell> ls

Parsed Tokens:
Token 1 : ls

Command Structure:
Command : ls
No Arguments
-----
MiniShell> ls -l

Parsed Tokens:
Token 1 : ls
Token 2 : -l

Command Structure:
Command : ls
Arguments:
Arg 1 : -l
-----
MiniShell> |

```